1.

a. Precedencia determia qual operador é aplicado primeiro quando operadores diferentes aparecem numa mesma expressão sem parênteses, enquanto associatividade determina a ordem de aokicação entre operadores de mesma precedência. Na expressão `c = a + (b = a + 5) * b + a`, os parênteses forçam `(b = a + 5)` a ser avaliado antes de qualquer outra coisa, independentemente de precedência; o resultado dessa subexpressão (`7`, já que `a = 2`) é então multiplicado por b, cujo valor, nesse ponto, já foi alterado pelo efeito colateral da atribuição interna, passando também a ser `7`, dando 49. A associatividade à esquerda de + faz a expressão ser avaliada como `(a + 49) + a`, resultando em 53. O resultado depende diretamente de b já estar atualizado no momento em que é lido novamente na multiplicação.

b. Há dois efeitos colaterais: a atribuição `b = a + 5`, que altera o valor de b como parte da avaliação de uma expressão (e não de um comando independente), e o uso subsequente desse b já modificado na mesma expressão maior `(* b)`, que é um efeito colateral de segunda ordem, o resultado da expressão inteira passa a depender da ordem em que suas partes são avaliadas.

c. Transparência referencial é a propriedade pela qual uma expressão pode ser substituída pelo seu valor sem alterar o comportamento do programa, o que só é garantido quando a expressão não possui efeitos colaterais. Quando há efeitos colaterais (como no exemplo acima), essa propriedade é quebrada: o valor de b dentro da expressão passa a depender de quando exatamente ele é avaliado, não apenas do seu valor "matemático". A ausência de transparência referencial prejudica a legibilidade (o leitor precisa simular mentalmente a ordem de avaliação para entender o resultado) e a confiabilidade (o mesmo trecho de código pode se comportar de forma diferente dependendo de detalhes de implementação do compilador), embora possa, em alguns casos, favorecer a capacidade de escrita ao permitir expressões mais compactas.

d. Quando uma expressão tem operandos com efeitos colaterais (como atribuições ou chamadas de função que alteram estado), a ordem em que os operandos são avaliados pode alterar o resultado final. No exemplo, se b fosse lido antes da atribuição `(b = a + 5)` ser executada, o resultado da multiplicação usaria o valor antigo de `b (3)`, produzindo `7 * 3 = 21` em vez de 49, um resultado completamente diferente para o mesmo texto de programa, dependendo apenas da ordem de avaliação escolhida pela implementação.

e. Algumas linguagens deixam a ordem de avaliação não especificada para dar liberdade ao compilador de reordenar ou paralelizar a avaliação de operandos visando otimizações de desempenho, sem ficar preso a uma ordem fixa imposta pela especificação da linguagem. O custo dessa liberdade é justamente a perda de previsibilidade em expressões com efeitos colaterais,

como discutido nos itens anteriores.

f. Overflow ocorre quando o resultado de uma operação excede a faixa representável pelo tipo (ex.: somar dois int no limite máximo); underflow, quando o resultado é menor do que o tipo consegue representar com precisão (comum em ponto flutuante, perto de zero); e divisão por zero é indefinida matematicamente e, dependendo do tipo, pode gerar comportamento indefinido, uma exceção, ou um valor especial (como `Infinity/NaN` em ponto flutuante). As linguagens tratam esses casos de formas distintas: interromper a execução com erro fatal, lançar uma exceção capturável, ou retornar um valor especial sem interromper o fluxo, cada abordagem representa um trade-off diferente entre confiabilidade (falhar de forma explícita e cedo) e robustez ou continuidade de execução.

2.

a. Expressões de tipo misto são aquelas em que os operandos envolvidos possuem tipos diferentes entre si, como somar um int com um float.

b. Para que uma expressão de tipo misto seja avaliada, é necessário que ao menos um dos operandos seja convertido para o tipo do outro (ou para um tipo comum) antes da operação, essa conversão automática, realizada implicitamente pela linguagem, é exatamente a coerção. Expressões de tipo misto, portanto, dependem das regras de coerção definidas pela linguagem para serem resolvidas.

c. Coerções em expressões de tipo misto podem prejudicar a legibilidade, pois o leitor precisa conhecer de cor as regras implícitas de conversão da linguagem para prever o tipo e a precisão do resultado, e podem prejudicar a confiabilidade quando a conversão implica perda de informação ou quando o comportamento não é óbvio a partir do código (ex.: divisão inteira quando pelo menos um operando não é convertido a tempo).

d. Sim, mas apenas entre tipos numéricos, com coerção implícita segura e unidirecional (int → float → double → decimal, sempre alargando a representação, nunca estreitando). Misturar str com tipo numérico, por exemplo, exige conversão explícita. Essa regra já havia sido definida e se mantém aqui: equilibra escrevibilidade (evita conversões manuais o tempo todo em contas simples) sem abrir mão de confiabilidade (nunca perde precisão silenciosamente).

3.

a. Sobrecarga de operadores é o mecanismo que permite que um mesmo símbolo de operador (como +) tenha significados diferentes dependendo dos tipos dos operandos envolvidos — por exemplo, + somando números ou concatenando strings.

b. Como vantagem, a sobrecarga permite notação mais natural e concisa para tipos definidos pelo usuário (ex.: somar vetores com +), melhorando a escrevibilidade e a legibilidade quando usada de forma consistente com a

semântica matemática esperada. Como risco, o uso indevido pode tornar o comportamento do operador imprevisível ou contraintuitivo (ex.: + fazendo algo sem relação alguma com soma), prejudicando a legibilidade e aumentando a chance de erros de interpretação por parte de quem lê o código.

c. Terá sobrecarga limitada, restrita a operadores de tipos estruturados nativos, por exemplo, + para concatenar str ou para unir List. Não haverá sobrecarga de operadores para struct definidos pelo usuário nesta primeira versão, evitando o risco de significados arbitrários e imprevisíveis (ex.: + sem relação com soma/concatenação), o que prioriza legibilidade sobre a expressividade adicional que a sobrecarga irrestrita traria.

d. Operadores de curto-circuito avaliam o segundo operando apenas se o primeiro não for suficiente para determinar o resultado da expressão, por exemplo, em A e B, se A for falso, B não é avaliado, pois o resultado já é falso independentemente do valor de B.

e. Isso evita efeitos colaterais indesejados quando a avaliação do segundo operando não deveria ocorrer (por exemplo, uma chamada de função com efeito colateral que só faz sentido se a primeira condição for verdadeira) e evita erros em tempo de execução em padrões comuns como `x != null e x.campo == valor`, em que avaliar o segundo operando sem a proteção do primeiro causaria um erro caso x não exista.

f. Sim, os operadores lógicos `e/ou (ou &&/||)` terão curto-circuito. Isso é especialmente importante combinado com Optional/Result: permite padrões como "verifique se o valor existe e então acesse um campo dele" sem risco de acessar um valor ausente, evitando erros em tempo de execução e efeitos colaterais desnecessários quando a primeira condição já basta para decidir o resultado.

4.

a. As principais formas são: atribuição simples `(x = valor)`, atribuição composta (`x += valor`, equivalente a `x = x + valor`), atribuição múltipla (várias variáveis recebendo o mesmo valor em um só comando) e atribuição por desestruturação (extraindo múltiplos valores de uma estrutura para variáveis distintas de uma vez).

b. Quando a atribuição é tratada como expressão (como em C, retornando o valor atribuído), ela pode ser aninhada dentro de outras expressões, como na questão 1, o que aumenta a expressividade, mas introduz efeitos colaterais estruturais dentro de expressões que, à primeira vista, pareceriam apenas calcular um valor, prejudicando a legibilidade. Quando a atribuição é tratada estritamente como comando (não retorna valor utilizável em expressões), essa possibilidade é eliminada por construção, tornando o código mais

previsível, ainda que ligeiramente menos conciso em certos padrões (como atribuições encadeadas)

c. Apenas atribuição simples (`=`), e só para variáveis explicitamente declaradas como mutáveis (já que o padrão é imutável); não haverá atribuição como expressão, atribuição múltipla nem encadeada, para manter cada mudança de estado como um comando isolado e visível no texto do programa.

5.

a. Desvios incondicionais (goto) permitem transferir o fluxo de execução para qualquer ponto do programa, sem qualquer estrutura hierárquica. Isso compromete fortemente a legibilidade, pois o fluxo de controle deixa de seguir a estrutura textual/visual do código, dificultando tanto a leitura quanto a verificação formal de corretude do programa.

b. `if-then-else` avalia uma condição booleana e desvia para um entre dois ramos possíveis, podendo ser encadeado (`else if`) para múltiplas condições independentes. `switch-case` compara um único valor/expressão contra uma série de casos possíveis, sendo mais legível e direto quando há muitas alternativas discretas baseadas no mesmo valor, mas menos flexível que `if-then-else` para condições arbitrárias e compostas.

c. O problema do dangling else ocorre quando um `if` aninhado sem chaves/delimitadores explícitos deixa ambíguo a qual `if` um `else` pertence, como em `if (A) if (B) X; else Y`; o else poderia, textualmente, pertencer tanto ao `if (A)` quanto ao `if (B)`. A maioria das linguagens resolve essa ambiguidade associando o `else ao if` mais próximo (mais interno), mas isso exige que o programador conheça essa regra, o que pode gerar leituras equivocadas caso a indentação sugira outra estrutura.

d. `while` é uma estrutura de repetição controlada por condição lógica, testada antes de cada iteração (pré-teste), podendo nunca executar o corpo. `do-while` também é controlada por condição lógica, mas testada depois de cada iteração (pós-teste), garantindo ao menos uma execução do corpo. `for` é tipicamente usada para repetição controlada por contador, combinando inicialização, condição de parada e incremento em uma única estrutura sintática, o que aumenta a legibilidade para esse padrão específico, embora também possa ser adaptada para condições lógicas gerais em várias linguagens.

e. A ApiLang terá `if-then-else, for` (controlado por contador/intervalo), `foreach` (iteração sobre List/Set/Map) e `pattern matching (match)` para desvio baseado em variantes de enum/Result/Optional. Não haverá `goto` nem `while` genérico: iterações são sempre limitadas a estruturas com fim previsível (coleção finita ou intervalo definido), reduzindo o risco de laços infinitos acidentais em serviços de API. Todo bloco de

`if`/`for`/`foreach`/`match` exige chaves `{ }` obrigatórias, o que elimina estruturalmente o problema do dangling else.

6.

a. Subprogramas (procedimentos e funções) são unidades nomeadas de código que encapsulam uma sequência de comandos reutilizável, recebendo parâmetros e (opcionalmente) retornando um valor. Sua função central na abstração é permitir que o programador raciocine sobre "o que" o subprograma faz, sem precisar reconsiderar "como" ele faz isso a cada uso, reduzindo complexidade e duplicação de código.

b. Funções são subprogramas que retornam um valor e tipicamente são usadas dentro de expressões, aproximando-se do conceito matemático de função; procedimentos (ou subprogramas void) executam uma sequência de ações, geralmente por seus efeitos colaterais (alterar estado, realizar E/S), sem retornar um valor a ser usado em uma expressão.

c. Subprogramas são declarados com `def nome(parametros): TipoRetorno { ... }`, sendo privados por padrão (exigindo uma palavra-chave explícita, como `pub`, para serem visíveis fora do módulo). Endpoints são um caso especial de subprograma: anônimos, associados a uma rota via decorator, e sempre públicos por definição, já que existem para serem expostos externamente.